

TECHNOLOGY STACK REFERENCE

ShowSzn

Every technology used in this project — versioned, categorized, and explained

Scope: Backend runtime, frontend runtime, data & caching layer, infrastructure, external services

Generated: August 2026

Repository: ~/projects/showszn

Contents

1. Full Stack Hierarchy (at a glance)

2. Backend

- 2.1 Language & Runtime — Java
- 2.2 Application Framework — Spring Boot & Spring Framework
- 2.3 Web Layer — Spring MVC & Embedded Tomcat
- 2.4 Data Layer — JPA, Hibernate, MySQL, HikariCP, Flyway
- 2.5 Caching & Concurrency — Redis, Spring Data Redis, Spring Cache
- 2.6 Security & Authentication — Spring Security, OAuth2, Clerk
- 2.7 Serialization — Jackson 3
- 2.8 Validation — Jakarta Bean Validation
- 2.9 Developer Tooling — Lombok, Maven

3. Frontend

- 3.1 Language — TypeScript
- 3.2 UI Library — React
- 3.3 Build Tooling — Vite
- 3.4 Routing — React Router
- 3.5 Server State — TanStack Query
- 3.6 HTTP Client — Axios
- 3.7 Styling — Tailwind CSS
- 3.8 Authentication SDK — Clerk React
- 3.9 Linting — oxlint

4. Infrastructure

- 4.1 Containerization — Docker & Docker Compose
- 4.2 Database Container
- 4.3 Cache Container

5. External Services

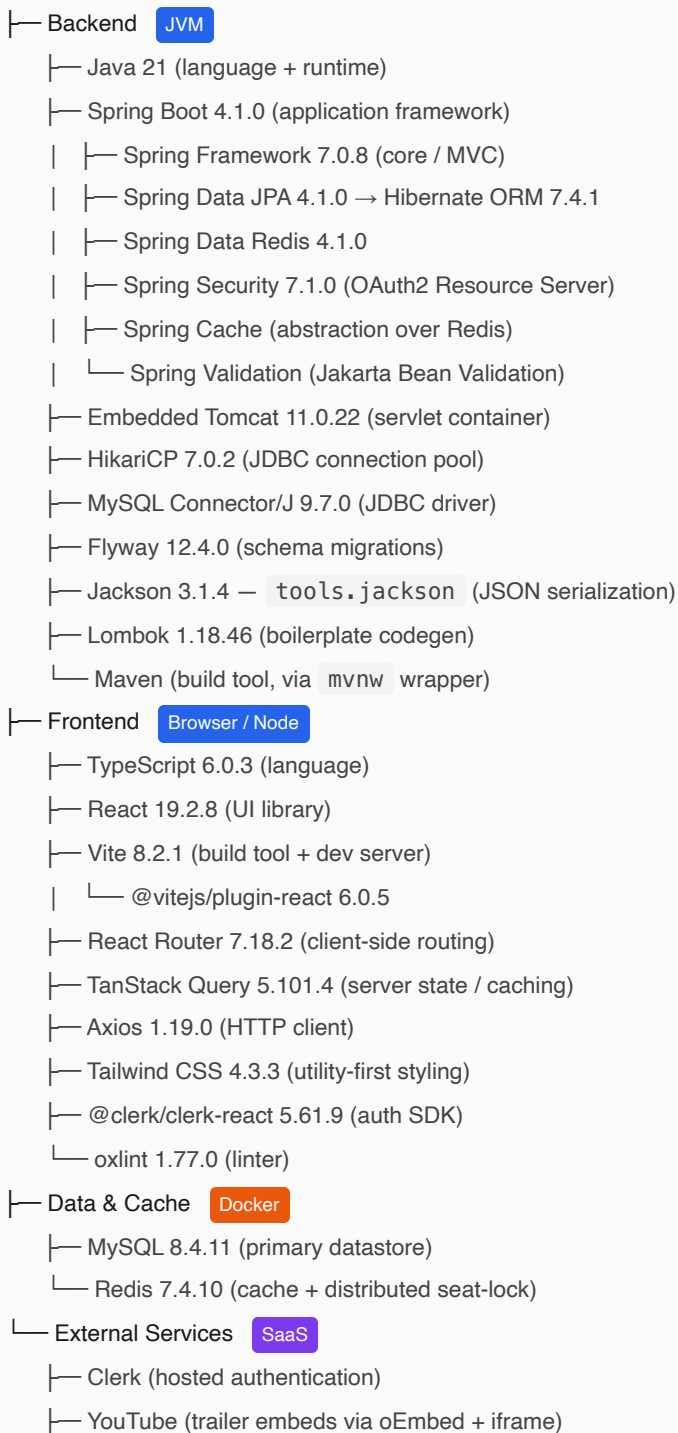
- 5.1 Clerk (Authentication-as-a-Service)
- 5.2 YouTube (oEmbed & iframe embeds)
- 5.3 Stripe (planned, not yet integrated)

6. Architectural Patterns Employed

1. Full Stack Hierarchy

A single-glance map of every layer, before the detailed sections below.

ShowSzn



└ Stripe (payments — planned, not yet wired)

2. Backend

2.1 Language & Runtime

Java **21.0.10 (OpenJDK)**

LANGUAGE / RUNTIME

The programming language and JVM the entire backend runs on. Java 21 is a Long-Term Support (LTS) release.

Why here: LTS stability plus access to modern language features used throughout the codebase — records (all DTOs are Java `record` types), pattern matching in `switch` expressions (seat-type price multipliers), and virtual-thread-ready tooling. Spring Boot 4 requires Java 17+; 21 was chosen as the current LTS.

2.2 Application Framework

Spring Boot **4.1.0**

APPLICATION FRAMEWORK

Opinionated auto-configuration layer over the Spring Framework. Handles dependency injection wiring, embedded server startup, externalized configuration (`application.yml`), and auto-configures every starter below based on what's on the classpath.

Why here: Convention-over-configuration meant each new domain (catalog → booking → events) could be added as a self-contained package with almost no manual wiring — a new `@Service` / `@RestController` / `@Repository` just works once on the classpath.

Spring Framework (Core, Context, MVC) **7.0.8**

UNDERLYING FRAMEWORK

The IoC container and web MVC framework that Spring Boot builds on top of. Provides `@Component` / `@Service` / `@RestController` stereotypes, the `DispatcherServlet` request-handling pipeline, and constructor-based dependency injection used throughout every service class in this project.

2.3 Web Layer

Spring Web MVC (*spring-boot-starter-webmvc*)

REST API LAYER

Provides `@RestController`, request/response body JSON binding, path/query parameter binding (`@PathVariable`, `@RequestParam`), and the `Pageable` / `Page<T>` abstraction used by every admin list endpoint.

Apache Tomcat (embedded) 11.0.22

SERVLET CONTAINER

The HTTP server embedded inside the Spring Boot application itself — no separate application server deployment needed. Listens on port 8080 in local development.

2.4 Data Layer

Spring Data JPA 4.1.0

PERSISTENCE ABSTRACTION

Repository-pattern abstraction over JPA. Every entity in this project (`Movie` , `Show` , `Booking` , `Event` , etc.) has a corresponding `JpaRepository` interface — Spring generates the implementation at runtime, including derived query methods (e.g. `findBySlug`) and custom `@Query` JPQL for join-fetch queries.

Why here: Every "lazy-loading masked as 401" bug documented in this project's build log was fixed using this layer's `join fetch` JPQL syntax — the single most-used feature of this dependency in practice.

Hibernate ORM 7.4.1.Final

JPA IMPLEMENTATION

The actual ORM engine behind Spring Data JPA — translates entity operations into SQL, manages the persistence context (first-level cache), and enforces the entity-to-table mapping declared via `@Entity` / `@Table` / `@Column` annotations. Configured with `ddl-auto: validate` , meaning Hibernate never generates schema itself — it only verifies that every entity matches what Flyway already created.

MySQL 8.4.11

PRIMARY DATABASE (RELATIONAL)

The system of record for every table: catalog (city/venue/screen/seat/movie/show), users, bookings, and events. Runs in Docker locally. Migrated onto mid-project from PostgreSQL.

Why here: Explicit user request to migrate off Postgres. Required rewriting all schema/seed SQL — `BIGSERIAL` → `AUTO_INCREMENT` , `generate_series()` → `WITH RECURSIVE CTEs`, and renaming the reserved word `show` to `showtime` .

MySQL Connector/J 9.7.0

JDBC DRIVER

The JDBC driver that lets the JVM talk to MySQL. Connection URL is configured with `serverTimezone=UTC` so that `DATETIME` columns round-trip consistently with Java's `Instant` type.

HikariCP 7.0.2

CONNECTION POOL

Spring Boot's default JDBC connection pool — pre-allocates and reuses database connections rather than opening a new TCP connection per request.

Flyway 12.4.0 (flyway-mysql)

SCHEMA MIGRATION TOOL

Version-controlled SQL migrations, run automatically on application startup. Every schema change and every batch of seed data in this project is a numbered file under `src/main/resources/db/migration/` (V1 through V7 at last count) — nothing is ever hand-run against the database outside of this mechanism.

Why here: Guarantees the schema is reproducible from a clean database in seconds (verified: ~2.25 seconds for the full catalog + ~255K seed rows) and that every environment (a fresh clone of this repo) ends up with an identical schema.

2.5 Caching & Concurrency

Redis 7.4.10

IN-MEMORY STORE — CACHE AND DISTRIBUTED LOCK

Used for two distinct purposes in this project: (1) HTTP response caching for hot public reads (city list, movies-by-city, events-by-city), and (2) the seat-hold mechanism for booking — `SETNX` with a 5-minute TTL guarantees two users can never hold the same seat simultaneously, and expired holds clean themselves up with no background job required.

Spring Data Redis 4.1.0

REDIS CLIENT ABSTRACTION

Provides `StringRedisTemplate` (used directly by `SeatLockService` for atomic hold acquisition) and the `RedisCacheManager` that backs the `@Cacheable` annotations.

Spring Cache Abstraction (*spring-boot-starter-cache*)

DECLARATIVE CACHING

`@Cacheable` / `@CacheEvict` annotations on service methods (e.g. `CatalogService.listCities()`). A custom `CacheConfig` registers a per-cache-name, explicitly-typed `JacksonJsonRedisSerializer` for each cache — necessary because the default generic serializer loses `List<T>` element type information on deserialize (documented as a real bug found and fixed in this project).

2.6 Security & Authentication

Spring Security 7.1.0

AUTHORIZATION & ROUTE PROTECTION

Configures three tiers of route access: `/api/public/**` (open), `/api/admin/**` (requires `ROLE_ADMIN`), everything else (requires any authenticated user). A custom `ClerkJwtAuthenticationConverter` enriches the authenticated principal with `ROLE_ADMIN` by looking up the requesting user in the database on every request.

Spring Security OAuth2 Resource Server

JWT VALIDATION

Validates every incoming JWT against Clerk's published JWKS (fetched from `{CLERK_ISSUER}/.well-known/openid-configuration`), lazily and cached — this project never issues its own tokens, it only verifies tokens Clerk already issued.

Clerk

EXTERNAL AUTHENTICATION PROVIDER

See §5.1 for full detail — included here because it is structurally part of the backend's security configuration (the JWT issuer) as well as a standalone external service.

2.7 Serialization

Jackson 3.1.4 — tools.jackson

JSON SERIALIZATION

Converts Java objects (DTO records) to/from JSON for every REST response and for the Redis-cached values. This is **Jackson 3**, not the more commonly-known Jackson 2 — the Maven coordinates moved from `com.fasterxml.jackson.*` to `tools.jackson.*`, which meant the classic Spring Data Redis serializer (`GenericJackson2JsonRedisSerializer`) doesn't exist on this classpath at all; the project uses its Jackson-3-compatible replacement, `GenericJacksonJsonRedisSerializer` (no "2"), instead.

2.8 Validation

Jakarta Bean Validation (*spring-boot-starter-validation*)

REQUEST VALIDATION

`@NotBlank`, `@NotNull`, `@Positive`, `@NotEmpty` annotations on every admin request DTO (`MovieRequest`, `EventRequest`, `HoldRequest`, etc.), enforced automatically via `@Valid` on controller method parameters.

2.9 Developer Tooling

Lombok `1.18.46`

BOILERPLATE CODE GENERATION

Compile-time annotation processor. Every JPA entity in this project uses `@Getter` `@Setter` `@NoArgsConstructor` `@AllArgsConstructor` `@Builder` instead of hand-written accessors — required because JPA entities need a no-args constructor, but the rest of the codebase wants to construct them via a fluent builder.

Maven (via the `mvnw` wrapper)

BUILD TOOL

Dependency resolution and build lifecycle. The wrapper script (`mvnw`) is committed to the repo so the exact Maven version is pinned — no separate Maven installation is required on a new machine.

3. Frontend

3.1 Language

TypeScript 6.0.3

LANGUAGE

A typed superset of JavaScript. Every API response shape in this project has a corresponding interface in `types.ts` (e.g. `MovieDetail`, `ShowSeatMap`, `BookingResponse`), which is what caught several would-be runtime bugs at compile time during development (stale field names after backend schema changes, for instance).

3.2 UI Library

React 19.2.8

UI LIBRARY

Component-based UI rendering. Every page (`HomePage`, `SeatSelectionPage`, admin pages, etc.) is a function component using hooks (`useState`, `useMemo`, `useEffect`) — no class components anywhere in the codebase.

3.3 Build Tooling

Vite 8.2.1

BUILD TOOL & DEV SERVER

Provides the dev server (with hot module replacement — most edits in this project reflected in the browser in under a second without a manual refresh) and the production build. Configured with a proxy that forwards `/api/*` requests to the backend on `:8080`, so the frontend dev server and backend can be developed against each other with no CORS configuration needed in dev.

@vitejs/plugin-react 6.0.5

VITE PLUGIN

Enables JSX/TSX compilation and React Fast Refresh inside Vite.

3.4 Routing

React Router 7.18.2

CLIENT-SIDE ROUTING

Declarative route definitions in `App.tsx` covering public pages, seat-selection/booking pages, and a nested `/admin/*` route tree with its own layout (`AdminLayout`) and shared access-gating.

3.5 Server State Management

TanStack Query 5.10.1.4

SERVER STATE / DATA FETCHING

Every piece of data that comes from the backend is fetched and cached via a `useQuery` hook, organized into domain-specific hook files (`queries.ts` , `eventQueries.ts` , `bookingQueries.ts` , `adminQueries.ts`). Mutations (`useMutation`) invalidate the relevant query keys on success rather than manually patching client-side cache.

Why here: Eliminates hand-written loading/error/cache state for every fetch — every page in this project gets loading skeletons and error states for free from the same hook that fetches the data.

3.6 HTTP Client

Axios 1.19.0

HTTP CLIENT

A single shared instance (`lib/api.ts`) with base URL `/api` . A request interceptor (`useAuthTokenInjector`) attaches the current Clerk session JWT as a Bearer token to every outgoing request.

3.7 Styling

Tailwind CSS 4.3.3

UTILITY-FIRST CSS FRAMEWORK

All styling in this project is done via utility classes directly in JSX — there is no separate CSS-in-JS or component stylesheet system. Custom keyframe animations (fade-in, scale-in) are defined once in `index.css` and reused as utility classes across the app.

3.8 Authentication SDK

@clerk/clerk-react 5.61.9

AUTH UI COMPONENTS + SESSION MANAGEMENT

Provides `<ClerkProvider>` , `<SignInButton>` , `<SignedIn>` / `<SignedOut>` conditional rendering, `<UserButton>` , and the `useAuth()` / `getToken()` hooks used to retrieve the session JWT for API calls.

3.9 Linting

oxlint **1.77.0**

LINTER

A Rust-based JavaScript/TypeScript linter, configured as the project's `npm run lint` script. Chosen over ESLint primarily for startup/run speed.

4. Infrastructure

4.1 Containerization

Docker & Docker Compose

LOCAL INFRASTRUCTURE

`docker-compose.yml` at the repo root defines two services — MySQL and Redis — so the entire data layer for local development starts with a single `docker compose up -d`. Neither the application backend nor frontend are containerized in this project; both run directly via `mvnw` / `npm run dev`.

4.2 Database Container

mysql:8.4 (Docker image)

CONTAINERIZED MYSQL

Runs on port 3306, persisted to a named Docker volume (`showsxn-mysql-data`) so data survives container restarts. Healthcheck via `mysqladmin ping`.

4.3 Cache Container

redis:7-alpine (Docker image)

CONTAINERIZED REDIS

Runs on port 6379. Alpine-based image for a smaller footprint. No persistence volume configured — acceptable because everything stored in Redis in this project (cache entries, seat holds) is either short-lived (5-minute TTL) or safely re-derivable from MySQL on a cache miss.

5. External Services

5.1 Clerk

Clerk

AUTHENTICATION-AS-A-SERVICE

Hosted sign-up/sign-in, session management, and JWT issuance. This project never stores a password or implements its own auth flow — the frontend embeds Clerk's hosted UI components, and the backend only verifies tokens Clerk already issued (see §2.6). A custom session-token template is configured in the Clerk dashboard to include `email` and `name` claims, since Clerk's default token omits them.

5.2 YouTube

YouTube (oEmbed API + iframe embeds)

TRAILER CONTENT DELIVERY

Movie trailers are real, openly-licensed short films (Blender Foundation / Blender Studio open movies), embedded via `youtube-nocookie.com/embed/{videoId}` iframes in a modal. Each video ID was verified via YouTube's oEmbed endpoint (`youtube.com/oembed?url=...`) before use, confirming title, channel, and embeddability.

5.3 Stripe (*planned*)

Stripe

PAYMENTS — NOT YET INTEGRATED

Environment variable placeholders (`STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET`) exist in `backend/.env.example` , but no Stripe SDK dependency has been added and no payment code exists yet. Booking confirmation currently finalizes immediately with no real charge (a mock-payment step) — see the separate technical documentation's "Known Limitations" section for the intended real-Stripe integration design (webhook-driven confirmation, not client-driven).

6. Architectural Patterns Employed

Not technologies in themselves, but recurring design patterns built on top of the technologies above — included here because they explain *how* several of the libraries above are actually used together.

Pattern	Where it's used
Layered architecture (Controller → Service → Repository)	Every domain package: <code>catalog</code> , <code>booking</code> , <code>event</code> , <code>eventbooking</code> , <code>admin</code>
Repository pattern	Every <code>*Repository</code> interface extending Spring Data's <code>JpaRepository</code>
DTO pattern (never expose entities directly)	Every controller returns a <code>record</code> DTO (e.g. <code>MovieSummaryResponse</code>), mapped from the JPA entity in a <code>from()</code> static factory method
Cache-aside	<code>@Cacheable</code> on <code>CatalogService</code> / <code>EventCatalogService</code> read methods, <code>@CacheEvict</code> on the admin mutations that could stale them
Distributed lock (Redis <code>SETNX</code>)	<code>SeatLockService</code> — the core correctness mechanism preventing double-booked seats
Namespaced key isolation	<code>SeatLockService</code> prefixes Redis keys with <code>movie:</code> / <code>event:</code> so two independent auto-increment ID sequences can never collide
JWT Bearer authentication (stateless)	Every authenticated API request; no server-side session state anywhere in the backend
Versioned database migrations	Every schema/seed change as a numbered Flyway file — never a manual ad-hoc SQL change
Shared presentational components over duplication	<code>SeatGrid</code> and <code>BookingConfirmedCard</code> components reused identically by both the movie and event booking flows

ShowSzn Technology Stack Reference — reflects the dependency versions actually resolved in this project at time of writing.